For the sciona project, we are reaching maturity on the atoms library and CDGs. We have 125 CDGs ingested representing every unique competition solution algorithm available on the open web as well as other open source academic/research repositories. We also have ingested the majority of the atoms needed to ground the CDG into a synthesizable, runnable algorithm.

The one exception is that until now we have stayed in the realm of conceptual tools grounded with reusable, generally cross-disciplinary atoms. We have a few CDGs (for example related to particle accelerators, or biomedical imaging analysis) that have atoms marked as "external knowledge". This means that without the knowledge of physics it would not be possible to create the algorithm. We are going to provide first-class support for physics.

By transitioning physics from siloed, jargon-heavy textbooks into normalized, composable "atoms," you are crossing the chasm from an AI coding assistant to an engine for **automated scientific discovery**.

The idea that an LLM might grab a fluid dynamics atom and apply it to algorithmic market liquidity—simply because the underlying mathematical topology of the problem matches—is the holy grail of algorithmic synthesis. You are essentially building a machine to automatically discover **isomorphisms** (mathematical equivalencies) between entirely disparate scientific fields.

Here is a breakdown of the best resources for ingesting, attributing, and contextualizing this knowledge, followed by architectural recommendations for how to structure these atoms so LLMs can synthesize them safely and creatively.

Part 1: Resources for Ingestion, Attribution, and Context

To build this library without manually scraping textbooks, you should leverage semantic web ontologies, open-source physics graphs, and datasets explicitly built for machine readability.

1. For Equations and "Alternate Uses":

- **Wikidata (via SPARQL):** Wikidata has an incredibly robust, heavily audited ontology for physics. If you query for "instances of physical equation" (Q23746617), you can extract thousands of formulas via the *defining formula* property (P2534).
    - *Crucially for your use case:* Wikidata utilizes the **"has use" (P366)** property. This will programmatically tell you that the Heat Equation (Q233938) has uses in "thermodynamics," "probability theory," and "financial mathematics" (Black-Scholes). This is the easiest way to map alternate uses.
- **The Physics Derivation Graph (PDG):** An ambitious open-source project attempting to document all mathematical physics as a directed graph. Rather than isolated equations, the PDG maps equations as nodes connected by "inference rules" that are pre-validated by Computer Algebra Systems (CAS). This mirrors your CDG architecture perfectly.
- **TheorIA & Phy-SRBench Datasets:** These are datasets built for ML symbolic regression. They contain fundamental equations, their variable definitions, and (most

importantly) the **empirical bounds/numerical ranges** for those variables. TheorIA strictly uses JSON schemas and embeds ORCID attributions for provenance.

2. For Empirical Constants & Spectra (Data Atoms):

- **NIST CODATA & DLMF:** The global standard for machine-readable physical constants and mathematical functions.
- **HITRAN & The Materials Project API:** The definitive open-source databases providing programmatic access to computed and experimental material properties, absorption spectra, and dielectric tensors.

3. For Cross-Disciplinary Mapping:

- **QUDT (Quantities, Units, Dimensions, and Types):** Originally sponsored by NASA, QUDT is the gold-standard semantic web ontology for physical dimensions. It maps the underlying dimensional constraints of variables across all sciences.
- **OPB (Ontology of Physics for Biology):** Because you are working with biosignals like PPG and EDA, the OPB is highly relevant. It is specifically designed to map classical physics, fluid dynamics, and thermodynamics to biological and physiological processes.

Part 2: Recommendations for "Doing This Well"

If physics equations are going to be ingested as CDG atoms for an LLM to synthesize, they require strict architectural constraints. Without them, LLMs are prone to hallucinate physically impossible algorithms (e.g., trying to add a temperature to a velocity).

1. The 3-Tier Decoupling (Logic vs. Grounding vs. Data)

To maximize cross-disciplinary discovery, separate the mathematical structure from the physical semantics and the empirical data.

- **The Math Atom (The Logic):** A generic differential equation (e.g., $y = a \cdot e^{-bx}$). It only knows about operators and variables.
- **The Physics Atom (The Grounding):** e.g., the *Beer-Lambert Law*. This points to the Math Atom but binds the variables to specific QUDT dimensions (Absorbance, Path Length, Molar Attenuation).
- **The Data Atom (The Constants):** e.g., *Published Melanin Absorption Spectra*.
- **Why?** If the LLM is trying to solve a problem in supply chain decay, it might not search for the "Beer-Lambert Law," but it will easily recognize the abstract *Math Atom* for exponential decay. Furthermore, decoupling the *Data Atom* means the LLM can take your exact blood pressure CDG, swap the "melanin" Data Atom for an "atmospheric aerosol" Data Atom, and instantly synthesize an algorithm for satellite-based greenhouse gas imaging.

2. Enforce Dimensional Analysis as a "Compiler Contract"

In standard programming, we use data types like int or float to prevent illegal operations. In physical algorithms, **dimensions are your type system**.

- Every input and output edge on a physics atom must be tagged with its fundamental dimensional signature (e.g., $M \cdot L^2 \cdot T^{-3}$ for Power).
- When the LLM connects two atoms in a CDG, use a Python library like pint or sympy.physics.units to perform automated dimensional analysis. If the units do not perfectly balance, the synthesis fails and prompts the LLM to self-correct the "dimensional mismatch."

3. Dejargonize via "System Dynamics" (Bond Graphs)

In engineering, there is a concept called **System Dynamics** which proves that electrical, mechanical, fluidic, and thermodynamic systems are mathematically identical at the structural level. They can all be reduced to generalized **"Effort"** (Voltage, Pressure, Force, Temperature) and **"Flow"** (Current, Fluid Flow, Velocity, Heat Transfer).

- Tag your equations using these generalized behavioral archetypes. If an LLM is tasked with "smoothing out fluctuations in a software data stream," tagging an equation as an *Effort-Flow Damper* allows the LLM to stumble upon an electrical RC-circuit equation, or a mechanical shock absorber equation, or a vascular Windkessel equation—recognizing that the underlying math is exactly what the software needs.

4. Explicit Regimes of Validity (Boundary Conditions)

In software, a sorting algorithm works regardless of the size of the list. In physics, equations are almost always approximations that break down at the extremes (e.g., Newtonian kinematics fails at the speed of light; Mie Scattering assumes particle size is roughly equal to the wavelength).

- Every physics atom must have an explicit validity_bounds parameter array.
- If the LLM feeds an atom inputs that violate these boundaries (e.g., passing sub-wavelength particle sizes into your Mie Scattering atom), the CDG engine should throw a mathematical warning: *"Assumptions violated: For $x \ll \lambda$, swap this for a Rayleigh Scattering atom."*

5. Store as Abstract Syntax Trees (ASTs), Not LaTeX

LaTeX is a presentation language; it only tells a computer how an equation *looks*.

- Ensure the mathematical core of your atoms are stored as Abstract Syntax Trees (ASTs), Content MathML, or SymPy objects. This guarantees that the algorithms generated from the CDGs are natively synthesizable in Python/C++, algebraically manipulable by the LLM, and fully differentiable if the algorithm requires gradient descent or optimization.

6. Generate the "Audit Graph"

Scientific algorithms require immense trust. Because you are pulling from external knowledge, the Sciona platform should treat provenance as a core feature. When the LLM outputs a novel

algorithm (like your PPG blood pressure tool), the platform should automatically traverse the CDG and generate a bibliography, citing the specific Wikidata Q-items, NIST CODATA dataset versions, and DOIs that mathematically ground the generated code.

Equation Representation

By using sympy strictly as your **Internal Representation (IR)** within the Sciona database, you unlock mathematical superpowers for the LLM, while still outputting a 100% pure NumPy script at the end. Think of SymPy as Sciona's compiler, and NumPy as the machine code.

Here are the profound benefits of this "Dual-State Architecture":

1. Omnidirectional Solving (The "One-to-Many" Atom)

A pure Python function is a one-way street. Take the Ideal Gas Law:

Python
```
def calc_pressure(n, R, T, V):
    return (n * R * T) / V
```

This atom is single-purpose. If the LLM is building a CDG where Pressure is known from a sensor and it needs to synthesize an algorithm to find Volume (V), it cannot use this atom. It either has to write a slow numerical root-finding loop or try to hallucinate the algebra to reverse it.

If stored as a SymPy expression: Eq(P * V, n * R * T), the LLM can programmatically ask Sciona to solve for *any* variable. sympy.solve(equation, V) perfectly outputs $V = \frac{nRT}{P}$. **One symbolic atom effectively represents every possible algebraic permutation.**

2. Cross-Atom Algebraic Simplification (Graph Collapse)

When an LLM connects multiple CDG atoms together, pure Python executes them sequentially as chained black boxes.

Imagine Atom A calculates $y = e^x$, and Atom B calculates $z = \ln(y)$.

- **Pure NumPy:** At runtime, the computer performs an expensive exponential calculation across millions of array elements, and then immediately performs an expensive logarithm. This wastes CPU cycles and accumulates floating-point rounding errors.
- **SymPy IR:** If Sciona composites the expressions symbolically *first*, it evaluates $\ln(e^x)$ and calls sympy.simplify(). The engine algebraically reduces the entire chain to simply $z = x$. You compress and optimize the chained physics atoms into their most elegant computational form **before** converting it to code.

3. Analytical Gradients (Optimization without PyTorch)

Many applied physics algorithms—like your PPG/EDA blood pressure algorithm—require curve fitting, gradient descent, or Kalman filters (sensor fusion). These require knowing the derivatives (Jacobians/Hessians) of the equations to update state estimates.

- With a pure NumPy function, optimization requires finite-difference approximations (adding tiny ϵ values to estimate slopes), which is computationally slow and mathematically noisy.
- SymPy calculates exact, analytical derivatives trivially (sympy.diff()). This allows your CDG engine to automatically generate the mathematically perfect gradient matrices necessary to optimize the very algorithms the LLM is building, all in raw NumPy.

4. Topological Discovery (Finding Isomorphisms)

In your previous message, you mentioned the excitement of an LLM discovering new applications for physics in disparate fields. To do this, the LLM needs to recognize that the mathematical *shape* of an algorithm matches a known physical law.

If an atom is just Python code, the LLM mostly sees syntax. If it is a SymPy expression, it is parsed into a strict Abstract Syntax Tree (AST). The LLM can strip away the variable names and look purely at the topological structure. It can deduce: *"Wait, the differential structure of this market liquidity problem is mathematically identical to the thermal diffusion atom."* It separates the logic from the domain jargon.

5. Automated Code Generation (Zero-Dependency Output)

You do not actually have to choose between SymPy and NumPy. SymPy is specifically designed to bridge this gap via its code printers.

The ideal Sciona pipeline looks like this:

1. **Storage:** Store the core equations as lightweight SymPy objects in your library.
2. **Synthesis:** The LLM links the atoms. Sciona uses SymPy in the background to dynamically rearrange the algebra, compute derivatives, check dimensional units, and simplify the math.
3. **Compilation:** Sciona passes the finalized AST to sympy.printing.numpy (or sympy.lambdify).
4. **Execution:** SymPy instantly translates the abstract mathematical tree into a **raw, pure, vectorized NumPy source code string**.

The final .py file you provide has zero sympy imports. It operates exactly as if a senior engineer wrote a highly optimized NumPy script by hand, but with the mathematical rigor guaranteed by a Computer Algebra System.

If an enterprise downloads an algorithm that dynamically uses SymPy to calculate algebra on the fly, SymPy becomes a runtime dependency. You would theoretically have to pay experts to audit millions of lines of SymPy's open-source library. That is impossible.

Instead, you use an **Ahead-of-Time (AOT) Compile-Before-Sign** architecture:

1. **Drafting (Tier 3):** The LLM conceptually maps the physics and rearranges the math using SymPy to create and ground the CDG.
2. **Compilation (System Level):** the synthesizer uses the converter to translate that math into pure NumPy. **SymPy is now entirely stripped out of the dependency tree.**
3. **The Audit (Tier 1):** The KYC-verified Expert claims the audit bounty. They are presented with the finalized, raw NumPy file. They audit the logic, verify the bounds, and—crucially as your plan states—**"attach their cryptographic signature strictly to the immutable hash" of the generated NumPy code.**